

PASO 1:

Importar librerías necesarias

```
In [2]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score
from sklearn.linear_model import LinearRegression
from colorama import Fore, init

init(autoreset=True, strip=False, convert=False)
```

PASO 2:

Cargar dataset de Redes Sociales

```
In [3]: # #TODO: Carga Los conjuntos de datos correspondientes:
data_set = pd.read_csv('redes_sociales.csv')
```

```
In [4]: data_set.info()
data_set.head()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   edad                  88 non-null    float64
 1   red_social_principal  91 non-null    str
 2   frecuencia_uso        90 non-null    str
 3   horas_diarias         94 non-null    float64
 4   sesiones_diarias      94 non-null    float64
 5   tipo_contenido        92 non-null    str
 6   interaccion_diaria    90 non-null    float64
 7   publica_contenido     90 non-null    str
 8   uso_nocturno          89 non-null    str
 9   notificaciones        87 non-null    str
10  motivo_principal      88 non-null    str
11  nivel_satisfaccion    89 non-null    str
dtypes: float64(4), str(8)
memory usage: 9.5 KB
```

Out[4]:

	edad	red_social_principal	frecuencia_uso	horas_diarias	sesiones_diarias	tipo_contenido
0	17.0	X	Raramente	0.8	1.0	Educación
1	36.0	YouTube	Varias veces al día	10.9	12.0	Tutoriales
2	27.0	YouTube	NaN	5.9	15.0	Memes
3	17.0	YouTube	Varias veces por semana	NaN	7.0	Deportes
4	29.0	Facebook	Raramente	0.9	1.0	Tutoriales

PASO3:

Identificar qué columnas contienen valores faltantes y su porcentaje

```
In [5]: # #TODO: Columnas que contienen valores faltantes:
print(Fore.YELLOW + "\nValores NULL por columna:")
print(data_set.isnull().sum())
```

Valores NULL por columna:

```
edad          12
red_social_principal    9
frecuencia_uso    10
horas_diarias     6
sesiones_diarias     6
tipo_contenido     8
interaccion_diaria    10
publica_contenido    10
uso_nocturno       11
notificaciones      13
motivo_principal    12
nivel_satisfaccion   11
dtype: int64
```

```
In [6]: # # TODO: Identificar y mostrar el porcentaje de datos faltantes:
#* porcentaje = (datos_faltantes / numero_filas) * 100
missing_percentage1 = round( (data_set.isnull().sum() / data_set.shape[0]) * 100, 2)

#* Imprimir Los porcentajes:
print(Fore.YELLOW + "Porcentaje de datos faltantes en dataset:")
print(missing_percentage1.to_string())
```

Porcentaje de datos faltantes en dataset:

edad	12.0
red_social_principal	9.0
frecuencia_uso	10.0
horas_diarias	6.0
sesiones_diarias	6.0
tipo_contenido	8.0
interaccion_diaria	10.0
publica_contenido	10.0
uso_nocturno	11.0
notificaciones	13.0
motivo_principal	12.0
nivel_satisfaccion	11.0

PASO 3:

Separar por categorias numericas y de Texto

```
In [7]: cat_col = data_set.select_dtypes(include=['object', 'str']).columns
num_col = data_set.select_dtypes(include=np.number).columns

print(Fore.CYAN + 'Categorical columns:\n', cat_col)
print(Fore.CYAN + 'Numerical columns:\n', num_col)
```

Categorical columns:

```
Index(['red_social_principal', 'frecuencia_uso', 'tipo_contenido',
      'publica_contenido', 'uso_nocturno', 'notificaciones',
      'motivo_principal', 'nivel_satisfaccion'],
      dtype='str')
```

Numerical columns:

```
Index(['edad', 'horas_diarias', 'sesiones_diarias', 'interaccion_diaria'], dtype='str')
```

PASO 4:

Reemplazar los datos faltantes del dataset:

- Categoricals por **moda**
- Numericos por **mediana**

Volver a comprobar los valores faltantes estén en 0%

```
In [8]: # #TODO: Iterar categorias y reemplazar los faltantes por la mode de las columnas
for col in cat_col:
    mode = data_set[col].mode()[0]
    data_set[col] = data_set[col].fillna(mode)

#TODO: Iterar categorias numericas y reemplazar los faltantes por la mediana de las
for col in num_col:
    median = data_set[col].median()
    data_set[col] = data_set[col].fillna(median)
```

```
print(Fore.YELLOW + "Porcentaje de datos faltantes en dataset:")
print( (round((data_set.isnull().sum() / data_set.shape[0]) * 100, 2) ).to_string()
```

Porcentaje de datos faltantes en dataset:

```
edad          0.0
red_social_principal  0.0
frecuencia_uso  0.0
horas_diarias  0.0
sesiones_diarias  0.0
tipo_contenido  0.0
interaccion_diaria  0.0
publica_contenido  0.0
uso_nocturno  0.0
notificaciones  0.0
motivo_principal  0.0
nivel_satisfaccion  0.0
```

PASO 5:

Define las variables:

- Variables de entrada -> X
- Variable de salida -> Y

```
In [9]: # ## Guarda una copia del DataFrame eliminando la columna 'publica_contenido'
x = data_set.drop('publica_contenido', axis=1)
## Guarda solo la columna 'publica_contenido' y mapear respuestas en 0 y 1
y = data_set['publica_contenido'].map({
    'No': 0,
    'Sí': 1
})

## Actualizar La Separar de columnas
cat_col = x.select_dtypes(include=['object', 'str']).columns
num_col = x.select_dtypes(include=np.number).columns

print(Fore.CYAN + "Categorías:", cat_col)
print(Fore.CYAN + "Numéricas:", num_col)
```

```
Categorías: Index(['red_social_principal', 'frecuencia_uso', 'tipo_contenido',
                  'uso_nocturno', 'notificaciones', 'motivo_principal',
                  'nivel_satisfaccion'],
                  dtype='str')
```

```
Numéricas: Index(['edad', 'horas_diarias', 'sesiones_diarias', 'interaccion_diaria'], dtype='str')
```

```
In [10]: # #TODO: Convertir variables categóricas a variables numéricas
## para trabajar solo con numeros en el entrenamiento
x = pd.get_dummies(x, drop_first=True)

x.head()
```

Out[10]:

	edad	horas_diarias	sesiones_diarias	interaccion_diaria	red_social_principal_Instagram	r
0	17.0	0.80	1.0	24.0	False	
1	36.0	10.90	12.0	117.0	False	
2	27.0	5.90	15.0	49.0	False	
3	17.0	4.75	7.0	23.0	False	
4	29.0	0.90	1.0	9.0	False	

5 rows × 36 columns



PASO 6:

Dividir el dataset en:

- 60 % entrenamiento
- 20 % validación
- 20 % prueba

Reproducir la misma división con "random_state=4"

```
In [11]: # #TODO: Dividir en entrenamiento (60%) y temporal (40%)
x_train, x_temp, y_train, y_temp = train_test_split(
    x, y,
    test_size=0.40,
    random_state=4
)

#TODO: Dividir el 40% restante en validación (20%) y prueba (20%)
x_val, x_test, y_val, y_test = train_test_split(
    x_temp, y_temp,
    test_size=0.50,
    random_state=4
)
```

```
In [12]: # #* Mostrar distribución de porcentajes
print(Fore.GREEN + "Entrenamiento:", x_train.shape[0])
print(Fore.GREEN + "Validación:", x_val.shape[0])
print(Fore.GREEN + "Prueba:", x_test.shape[0])
```

Entrenamiento: 60

Validación: 20

Prueba: 20

PASO 7:

Obtener:

- Matriz de confusión
- Porcentajes de Accuracy
- Precision
- Recall
- F1-Score

1. Crea y entrena un modelo de Regresión Lineal utilizando los datos de entrenamiento.

```
In [13]: # #TODO: Crear modelo:
model = LinearRegression()

#TODO: Entrenar el modelo
model.fit(x_train, y_train)

print(Fore.YELLOW + "Intersección:", model.intercept_)
print(Fore.YELLOW + "Coeficiente:\n", model.coef_)
```

Intersección: 0.2723387519912802

Coeficiente:

```
[-3.90742193e-03 -4.01428770e-02  3.16687304e-02  5.20343686e-04
 3.49933621e-01 -7.42885526e-02  1.65822674e-01  3.66355425e-02
 2.10665769e-01 -9.32660314e-01 -5.97112846e-02 -1.48665270e-01
-1.60414261e-01  6.39653972e-01  7.54552818e-01  4.42274877e-01
 6.22669883e-01  3.59162925e-01  7.46328893e-01  3.33176544e-01
-1.39808411e-01  4.86754968e-01 -5.70380954e-01  1.43240965e-02
-3.88647714e-01  2.38528904e-01 -2.70441409e-01  9.46616903e-02
-1.20696969e-01  1.04014424e-01 -1.27309368e-01  1.25727723e-01
-7.03432557e-01 -2.72182656e-01  1.92710788e-01 -2.24313297e-01]
```

2. Generar las predicciones con el modelo y convertirlas a 0 o 1 usando 0.5 como umbral.

```
In [14]: # #TODO: Obtener las predicciones
y_pred = model.predict(x_test)
print(y_pred)

#TODO: Convertir a 0 y 1
y_pred = (y_pred >= 0.5).astype(int)
print(y_pred)
```

```
[-1.3451832  1.25710529  0.44709994  0.74834596  0.09024891 -1.23341297
 0.55869403  0.49967909 -0.23952301  0.54191893 -0.76132519  1.00586584
 0.47503247  0.14599797 -0.98417227 -0.45769236 -0.23982582  0.68062405
 0.03707965  0.3568629 ]
[0 1 0 1 0 0 1 0 0 1 0 1 0 0 0 0 0 1 0 0]
```

3. Calcular las metricas para evaluar qué tan bien realiza las predicciones.

```
In [15]: # #* Matriz de confución:
matriz = confusion_matrix(y_test, y_pred)

#* Exactitud
accuracy = accuracy_score(y_test, y_pred)

#* Precisión
```

```
precision = precision_score(y_test, y_pred)
## Sensibilidad
recall = recall_score(y_test, y_pred)
## F1-Score
f1 = f1_score(y_test, y_pred)
```

```
In [16]: print(Fore.GREEN + "Matriz de confusión:\n", matriz)

print(Fore.GREEN + f"\nExactitud: {Fore.RESET}{round( accuracy * 100, 2)}%")
print(Fore.GREEN + f"Precisión: {Fore.RESET}{round( precision * 100, 2)}%")
print(Fore.GREEN + f"Sensibilidad: {Fore.RESET}{round( recall * 100, 2)}%")
print(Fore.GREEN + f"F1-Score: {Fore.RESET}{round( f1 * 100, 2)}%")
```

Precisión: 16.67%

```
[[9 5]
 [5 1]]
```

Exactitud: 50.0%

Precisión: 16.67%

Sensibilidad: 16.67%

F1-Score: 16.67%

PASO 8:

Problemas considerar en el dataset

1. Valores faltantes: Se identificaron y se reemplazaron.

- Variables categóricas por la moda de la columna.
- Variables numéricas por la mediana de la columna.

2. Variables categóricas: El modelo necesita valores numéricos, por lo que se utilizó

`pd.get_dummies()` para convertirlas.

3. Variable objetivo categórica: `publica_contenido` tenía Sí/No, por lo que se convirtió a:

- No = 0
- Sí = 1

4. Predicciones continuas: Debido a que se utilizó `LinearRegression` para predecir una variable binaria, el modelo genera valores continuos. Para convertir estas predicciones en las categorías 0 y 1, se estableció un umbral de 0.5.

5. Datos atípicos: Se consideró la posible presencia de valores atípicos, ya que pueden afectar el rendimiento del modelo. No se realizó una eliminación de estos datos.